

Recursion in C

Recursion is the process by which a function calls itself. C language allows writing of such functions which call itself to solve complicated problems by breaking them down into simple and easy problems. These functions are known as **recursive functions**.

What is a Recursive Function in C?

A recursive function in C is a **function** that calls itself. A recursive function is used when a certain problem is defined in terms of itself. Although it involves iteration, using iterative approach to solve such problems can be tedious. Recursive approach provides a very concise solution to seemingly complex problems.

Syntax

This is how a general recursive function looks like –

```
void recursive_function(){
    recursion();    // function calls itself
}

int main(){
    recursive_function();
}
```

While using recursion, programmers need to be careful to define an exit condition from the function, otherwise it will go into an **infinite loop**.

Why Recursion is Used in C?

Recursion is used to perform complex tasks such as **tree** and **graph** structure traversals. Popular recursive programming solutions include factorial, binary search, tree traversal, tower of Hanoi, eight queens problem in chess, etc.

A recursive program becomes concise, it is not easily comprehensible. Even if the size of the code may reduce, it needs more resources of the processor, as it involves multiple IO calls to the function.

Explore our **latest online courses** and learn new skills at your own pace. Enroll and become a certified expert to boost your career.

Factorial Using Recursion

Recursive functions are very useful to solve many mathematical problems such as calculating the factorial of a number, generating Fibonacci series, etc.

The most popular example of recursion is calculation of factorial. Mathematically, a factorial is defined as –

$$n! = n \times (n-1)!$$

It can be seen that we use factorial itself to define factorial. Hence this is a fit case to write a recursive function. Let us expand the above definition for calculating the factorial value of 5.

$$\begin{aligned} 5! &= 5 \times 4! \\ &= 5 \times 4 \times 3! \\ &= 5 \times 4 \times 3 \times 2! \\ &= 5 \times 4 \times 3 \times 2 \times 1! \\ &= 5 \times 4 \times 3 \times 2 \times 1 \\ &= 120 \end{aligned}$$

While we can perform this calculation using a loop, its recursive function involves successively calling it by decrementing the number till it reaches 1.

Example: Non-Recursive Factorial Function

The following program shows how you can use a non-recursive function to calculate the factorial of a number –

</>

Open Compiler

```
#include <stdio.h>
#include <math.h>

// function declaration
int factorial(int);

int main(){
    int a = 5;
    int f = factorial(a);

    printf("a: %d \n", a);
```

```
printf("Factorial of a: %d", f);
}
int factorial(int x){
    int i;
    int f = 1;

    for (i = 5; i >= 1; i--){
        f *= i;
    }
    return f;
}
```

Output

When you run this code, it will produce the following output –

```
a: 5
Factorial of a: 120
```

Example: Recursive Factorial Function

Let us now write a recursive function for calculating the factorial of a given number.

The following example calculates the factorial of a given number using a recursive function –


[Open Compiler](#)

```
#include <stdio.h>
#include <math.h>

/* function declaration */
int factorial(int i){

    if(i <= 1){
        return 1;
    }
    return i * factorial(i - 1);
}

int main(){
    int a = 5;
```

```
int f = factorial(a);

printf("a: %d \n", a);
printf("Factorial of a: %d", f);
return 0;
}
```

Output

Run the code and check its output –

```
a: 5
Factorial of a: 120
```

When the main() function calls the factorial() function by passing the variable "a", its value is stored in "i". The factorial() function successively calls itself.

In each call, the value of "i" is multiplied by its earlier value after reducing it by 1, till it reaches 1. As it reaches 1, the product of all the values between the initial value of the argument and 1 is returned to the main() function.

Binary Search Using Recursion

Let us have a look at another example to understand how recursion works. The problem at hand is to check whether a given number is present in an **array**.

While we can perform a **sequential search** for a certain number in the list using a **for loop** and comparing each number, the sequential search is not efficient, especially if the list is too long.

The **binary search algorithm** checks if the index "start" is greater than the index "end". Based on the value present at the variable "mid", the function is called again to search for the element.

We have a list of numbers arranged in ascending order. Then we find the midpoint of the list and restrict the checking to either left or right of the midpoint, depending on whether the desired number is less than or greater than the number at the midpoint.

Example: Recursive Binary Search

The following code implements the recursive binary searching technique –

```
</>
```

[Open Compiler](#)

```
#include <stdio.h>

int bSearch(int array[], int start, int end, int element){

    if (end >= start){

        int mid = start + (end - start ) / 2;

        if (array[mid] == element)
            return mid;

        if (array[mid] > element)
            return bSearch(array, start, mid-1, element);
        return bSearch(array, mid+1, end, element);
    }
    return -1;
}

int main(void){
    int array[] = {5, 12, 23, 45,    49, 67, 71, 77, 82};
    int n = 9;
    int element = 67;
    int index = bSearch(array, 0, n-1, element);

    if(index == -1 ){
        printf("Element not found in the array ");
    }
    else{
        printf("Element found at index: %d", index);
    }
    return 0;
}
```

Output

Run the code and check its output –

```
Element found at index: 5
```

Fibonacci Series Using Recursion

In Fibonacci series, a number is the sum of its previous two numbers. To generate Fibonacci series, the i th number is the addition of $i-1$ and $i-2$.

Example

The following example generates the first 10 numbers in the Fibonacci series for a given number using a recursive function –


[Open Compiler](#)

```
#include <stdio.h>

int fibonacci(int i){

    if(i == 0){
        return 0;
    }

    if(i == 1){
        return 1;
    }
    return fibonacci(i-1) + fibonacci(i-2);
}

int main(){

    int i;

    for (i = 0; i < 10; i++){

        printf("%d\t", fibonacci(i));

    }
    return 0;
}
```

Output

When the above code is compiled and executed, it produces the following result –

```

0
1
1
2
3
5
8
13
21
34

```

Implementing recursion in a program is difficult for beginners. While any iterative process can be converted in a recursive process, not all cases of recursion can be easily expressed iteratively.